Assignment 1: A Scalar Autograd Engine and Neural Network API

MicroGrad from scratch — design, derivations and results

CSE485/CSE488 Deep Learning, Ain Shams University

October 25, 2026

Abstract

This report documents the design and verification of a scalar automatic differentiation engine (`src/engine.py`) and a small neural-network API (`src/nn.py`) implemented from scratch in Python, without PyTorch or any other autodiff library. We derive every local backward rule by hand, explain reverse-mode autodiff and the role of the topological sort, and justify the implementation choices (closure-based backpropagation, set-based parent tracking, activation choice, weight initialisation, constant-exponent power). Verification is presented at three levels: the official test suite (**12/12 passing**), a hand-derived sanity check, and a finite-difference gradient check over a trained model whose analytic gradients agree with central differences to a maximum relative error of 2.96×10^{-9}. As an end-to-end demonstration, a 337-parameter multilayer perceptron trained with mean squared error and mini-batch SGD reaches **97.5%** accuracy on a two-moons classification task.

Contents

1 Overview and architecture **3**

2 The chain rule and reverse-mode autodiff **3**
2.1 Local derivatives (derived by hand) . . . 3

3 Topological sort and the backward pass **4**
3.1 Why a topological order is required . . . 4
3.2 Why post-order (append after the children loop) . . . 5
3.3 Worked example . . . 5

4 The diamond graph: `visited` and `+=` **5**
4.1 Why the `visited` set . . . 5
4.2 Why `+=` and not `=` . . . 6

5 Design justifications **6**

6 The neural-network API **7**
6.1 Neuron, Layer, MLP . . . 7
6.2 Loss and optimisation . . . 7

7 Verification **8**
7.1 Hand-derived sanity check . . . 8
7.2 Gradient checking . . . 8
7.3 Test suite: 12/12 . . . 9

8 Experimental results **9**

9 Answers to the conceptual questions **10**

10 Reproducibility **12**

11 Conclusion **12**

1 Overview and architecture

The assignment consists of two modules plus their verification material:

File	Role
`src/engine.py`	`Value`: scalar node with reverse-mode autodiff
`src/nn.py`	`Neuron`, `Layer`, `MLP` — PyTorch-like forward API
`tests/`	12 official tests (`pytest`)
`demo.ipynb`	reproducible training run, plots, gradient check
`report/`	this LaTeX report and its figures

A `Value` stores a scalar `data` together with the computational graph that produced it (`_prev` / `_op`), a running gradient `grad`, and a `_backward` closure implementing its local derivative. Every operation returns a fresh output node whose `_backward` closure knows nothing but the operands immediately below it; calling `Value.backward` walks the whole graph once in reverse topological order and applies the chain rule, so each node ends up holding $\partial L/\partial(\text{node})$.

Two properties of the representation do most of the work:

- **Graph capture for free.** Because Python evaluates `a * b` before calling `__mul__`, the operation can hand its inputs to the output node as children (`(self, other)`) without any explicit tracing or tape. Building the forward pass *is* building the graph.
- **Two-phase evaluation.** The forward pass only writes `data`; the backward pass only reads `data` and writes `grad`. No value read during differentiation can be clobbered by the differentiation itself.

2 The chain rule and reverse-mode autodiff

Let the loss be a composition of local functions

$$L = f_k \circ f_{k-1} \circ \cdots \circ f_1, \tag{1}$$

so that for an input x_i sitting under many intermediate values,

$$\frac{\mathrm{d}L}{\mathrm{d}x_i} = \sum_{\text{paths } p:\, x_i \to L} \prod_{e \in p} \frac{\partial \text{head}_e}{\partial \text{tail}_e}. \tag{2}$$

Equation (2) is a *sum over paths*, which is exactly why gradients must be accumulated rather than assigned (Section 4).

Reverse mode seeds $\partial L/\partial L = 1$ and propagates *downwards*, multiplying by one local derivative per edge. Its cost is one multiply–add per edge, independent of how many inputs x_i there are; forward mode would cost one sweep per input. Deep networks have millions of parameters and exactly one scalar loss, so reverse mode is the only practical choice — the same reason PyTorch uses it.

2.1 Local derivatives (derived by hand)

Each operation needs only its own derivative; the chain rule composes them.

Addition. $c = a + b$ with a, b scalars:

$$\frac{\partial c}{\partial a} = 1, \qquad \frac{\partial c}{\partial b} = 1.$$

The gradient is passed through unchanged to both parents.

Multiplication. $c = a \cdot b$:

$$\frac{\partial c}{\partial a} = b, \qquad \frac{\partial c}{\partial b} = a.$$

Each parent's share of the gradient is scaled by the *other* operand's forward value — which is why forward values must be retained.

Power. $c = a^k$, k a constant:

$$\frac{\partial c}{\partial a} = k\, a^{k-1}.$$

This holds only for constant k; a `Value` exponent would additionally contribute $a^k \ln a \cdot \partial k/\partial a$, which the local rule does not model. The implementation therefore asserts `isinstance(other, (int, float))` and fails fast rather than returning a silently wrong gradient.

Tanh. $t = \tanh(a)$, with the forward value t already computed:

$$\frac{\partial t}{\partial a} = 1 - \tanh^2(a) = 1 - t^2.$$

Reusing t means no transcendental function is evaluated in the backward pass.

Exp. $e = \exp(a)$:

$$\frac{\partial e}{\partial a} = \exp(a) = e.$$

Again the forward value is the derivative, so the backward pass is a copy.

The remaining operators are *derived*, not primitive: subtraction, division and negation are expressed through the rules above,

$$-a \equiv a \cdot (-1), \qquad a - b \equiv a + (-b), \qquad a/b \equiv a \cdot b^{-1},$$

so a single addition rule and a single multiplication rule cover all five.

3 Topological sort and the backward pass

3.1 Why a topological order is required

A node may only be backpropagated through *after* every consumer of its gradient has contributed. If we pushed a partial gradient into `a.grad` and ran `a`'s backward rule immediately, that rule would multiply the incomplete value by a local derivative, and later contributions from other paths would be scaled incorrectly. A topological order guarantees that when a node is processed, `grad` already holds its final value.

3.2 Why post-order (append after the children loop)

`_get_topo` is a depth-first search that appends a node *after* recursing into all of its children (post-order). Post-order therefore lists every node *after* its dependencies, i.e. children before parents. The backward pass then simply walks that list in reverse, starting from the output:

```
def backward(self):
    topo = self._get_topo()
    self.grad = 1.0
    for n in reversed(topo):
        n._backward()
```

Seeding the root with `self.grad = 1.0` encodes $\partial L/\partial L = 1$. It also means calling `.backward()` on an *intermediate* node yields that node's gradient with respect to the inputs, which is what makes intermediate-value tests possible.

3.3 Worked example

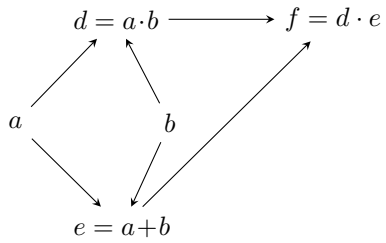
Take $a = -2$, $b = 3$,

$$d = a \cdot b, \qquad e = a + b, \qquad f = d \cdot e,$$

so $f = ab(a + b) = a^2b + ab^2$ and

$$\frac{\partial f}{\partial a} = 2ab + b^2 = 2(-2)(3) + 9 = -3, \qquad \frac{\partial f}{\partial b} = a^2 + 2ab = 4 + 2(-2)(3) = -8.$$

The graph has the shape



Post-order DFS gives `[a, b, d, e, f]`; reversed, the backward sweep processes $f \to e \to d \to b \to a$, and each node's `grad` is final when it is reached. This is precisely the template test `test_chain_rule_and_backprop` in `tests/test_engine.py`.

4 The diamond graph: `visited` and `+=`

4.1 Why the `visited` set

Real graphs share nodes: a feeds d and e, and d, e both feed f. Two things go wrong without bookkeeping:

1. **Repeated work.** Without a set, a is re-explored once per path reaching it — exponential in graph depth.
2. **Duplicate list entries.** a would be appended to `topo` several times, so `backward` would run its closure several times and push gradient through it several times.

`visited` (a `set`) is checked *before* recursing, so each node is appended exactly once. A `set` also matches the semantics of `_prev`: a node that appears twice in one expression, as in $a + a$, is one parent, not two independent ones.

4.2 Why `+=` and not `=`

Let $a = 3$, $c = a + a$, $d = c \cdot a$. Then $d = 2a \cdot a = 2a^2$ and $\mathrm{d}d/\mathrm{d}a = 4a = 12$.

Tracing by hand:

- d's rule pushes a's *direct* contribution $c = 6$;
- c's rule pushes its gradient ($= a = 3$) into a *twice*, because both operands are a: $+3$ and $+3$.

Total $6 + 3 + 3 = 12$. ✓

With assignment, whichever write executes last wins and the other contributions vanish, yielding a wrong number that depends on evaluation order. Since Equation (2) is a sum over *all* paths, so accumulation is not an optimisation — it is the chain rule.

The template test `test_gradient_accumulation_diamond_graph` asserts precisely this behaviour.

5 Design justifications

Why closures instead of an operator dispatch table? Each operation already knows its operands at construction time, so a closure captures them directly: no string-to-function lookup on `_op`, no re-derivation during the backward pass, and the backward rule lives next to the forward rule it belongs to. `_op` and `label` remain purely diagnostic.

Why `_prev` is a `set`? Parents are shared arbitrarily; a set deduplicates them for the topological walk and gives the natural semantics for repeated operands ($a + a$ has one parent). It also keeps `out` hashable-by-identity in `visited`.

Why is `grad` initialised to `0.0`? It makes "no effect yet" the default, so `+=` works from a clean slate and a gradient that is never written reads as zero rather than `None`. `reset_grad()` restores this state; `nn.py` exposes it as `zero_grad()` on the model, mirroring PyTorch and keeping optimisation steps independent of each other.

Why uniform weights in $[-1, 1]$? Initial pre-activations stay inside `tanh`'s near-linear region ($1 - \tanh^2 \approx 1$), so units are not born saturated and can learn from the first step. Random values also break the symmetry that would otherwise make every unit in a layer compute identical features.

Why `tanh` rather than sigmoid? Outputs are zero-centred in $(-1, 1)$, which gives better-conditioned gradients for the next layer, and the derivative $1 - t^2$ reuses the forward value. The cost is saturation: a confidently wrong unit has near-zero gradient (visible directly in Section 8).

Why is the power exponent restricted? Covered in Section 3.1: the rule ka^{k-1} is only correct for constant k, so an assert converts a silent wrong answer into a loud correct one.

Why no dependency on NumPy inside `src/`? The point of the exercise is to internalise computational graphs, local derivatives, topological ordering and accumulation rather than to obtain gradients from a library. NumPy appears only in `demo.ipynb`, for dataset generation and plotting.

6 The neural-network API

6.1 Neuron, Layer, MLP

```
class Neuron:
    def __init__(self, nin):
        self.w = [Value(random.uniform(-1, 1)) for _ in range(nin)]
        self.b = Value(random.uniform(-1, 1))

    def __call__(self, x):
        act = self.b
        for wi, xi in zip(self.w, x):
            act += wi * xi
        return act.tanh()
```

A neuron computes the affine form $z = \sum_i w_i x_i + b$ as a chain of `Value` operations and squashes it:

$$h = \tanh\Big(\sum_{i=1}^{n} w_i x_i + b\Big).$$

Because the forward pass is written with ordinary operators, the whole network is a computational graph on which `backward()` already works — the engine needs no knowledge of neural networks at all.

`Layer` holds `nout` neurons over the same `nin` inputs and returns a list, collapsing to a bare `Value` when `nout == 1`; `MLP` chains layers, feeding the outputs of layer j into layer $j+1$.

Parameter accounting for the demo model `MLP(2, [16, 16, 1])`:

$$\underbrace{(2+1)\cdot 16}_{48} + \underbrace{(16+1)\cdot 16}_{272} + \underbrace{(16+1)\cdot 1}_{17} = 337,$$

where each $+1$ accounts for a bias. `parameters()` flattens the layers in order, so the 337-element list and the notebook's parameter indices line up one-to-one; `zero_grad()` walks the same list.

6.2 Loss and optimisation

The demo minimises **mean squared error** between the network score $s_i \in (-1, 1)$ and the label $y_i \in \{-1, +1\}$:

$$L = \frac{1}{N}\sum_{i=1}^{N}(s_i - y_i)^2. \tag{3}$$

Equation (3) is built from `Value` operators only — a difference, a product, an accumulation and a scalar multiply — so its gradient flows back into every weight with no special-cased code.

Training is mini-batch SGD (batch size 20), with the update performed *after each batch*:

$$p \leftarrow p - \eta \frac{\partial L_{\text{batch}}}{\partial p}, \qquad \eta \leftarrow 0.98\,\eta \text{ per epoch}, \qquad \eta_0 = 0.1. \tag{4}$$

A step sequence per iteration is `model.zero_grad()` → `loss.backward()` → `p.data -= lr * p.grad`. Gradients are never stored across steps.

7 Verification

7.1 Hand-derived sanity check

The notebook opens with a two-variable expression whose gradients are derived on paper: $f(a, b) = a \cdot b + \tanh(a)$, giving $\partial f/\partial a = b + 1 - \tanh^2(a)$ and $\partial f/\partial b = a$. With $a = 0.5$, $b = -2$:

Quantity	Engine	Hand-derived
f	-0.537883	—
$\partial f/\partial a$	-1.213552	-1.213552
$\partial f/\partial b$	0.500000	0.500000

Both gradients match to 10^{-9}.

7.2 Gradient checking

After training, analytic gradients are compared against central differences,

$$\frac{\partial L}{\partial p} \approx \frac{L(p + h) - L(p - h)}{2h}, \qquad h = 10^{-5}, \tag{5}$$

for six randomly selected parameters of the trained model, recomputing the full mean-squared-error loss with the parameter nudged each time. The relative error is reported as

$$\frac{\left|g_{\text{analytic}} - g_{\text{numeric}}\right|}{\max\left(10^{-8},\ |g_{\text{analytic}}| + |g_{\text{numeric}}|\right)}.$$

Parameter index	Analytic	Central difference	Relative error
69	0.00906473	0.00906473	6.54×10^{-11}
130	0.00278148	0.00278148	3.91×10^{-10}
265	-0.00210914	-0.00210914	2.90×10^{-11}
278	-0.00187257	-0.00187257	1.65×10^{-10}
295	-0.00028168	-0.00028168	2.96×10^{-9}
316	0.00047361	0.00047361	3.64×10^{-10}
		Maximum relative error	$\mathbf{2.96 \times 10^{-9}}$

Agreement at the 10^{-9} level confirms the backward rules, the topological order and the accumulation logic simultaneously — an error in any one of them would show up as a discrepancy orders of magnitude larger. The notebook asserts a threshold of 10^{-4}, which is met with four orders of margin.

7.3 Test suite: 12/12

Test	Covers
`test_value_initialization`	`data`, `grad = 0.0`, `label`, `__repr__`
`test_basic_operations`	add, radd, mul, rmul, sub, div, neg
`test_type_errors`	`TypeError` on `+` with a string, `*` with a list
`test_power`	$a^3 \Rightarrow 12$; `AssertionError` on a `Value` exponent
`test_activations_and_exp`	`tanh` at 0, `exp` at 0 with gradients
`test_gradient_accumulation_diamond_graph`	$d = (a+a) \cdot a \Rightarrow 12$; catches `=` instead of `+=`
`test_chain_rule_and_backprop`	composite graph: $\partial f/\partial a = -3$, $\partial f/\partial b = -8$
`test_edge_case_division_by_zero`	`ZeroDivisionError`
`test_type_casting`	`__int__`, `__float__`
`test_neuron`	parameter shapes, tanh range, output
`test_layer`	sizes, list output, parameter count
`test_mlp`	depth, parameter count, scalar output

```
$ python -m pytest

............                                                    [100%]
12 passed in 0.01s
```

8 Experimental results

The demo trains `MLP(2, [16, 16, 1])` on a two-moons dataset generated in-notebook (200 samples, Gaussian noise $\sigma = 0.15$, labels ± 1, seed 42) — no scikit-learn dependency.

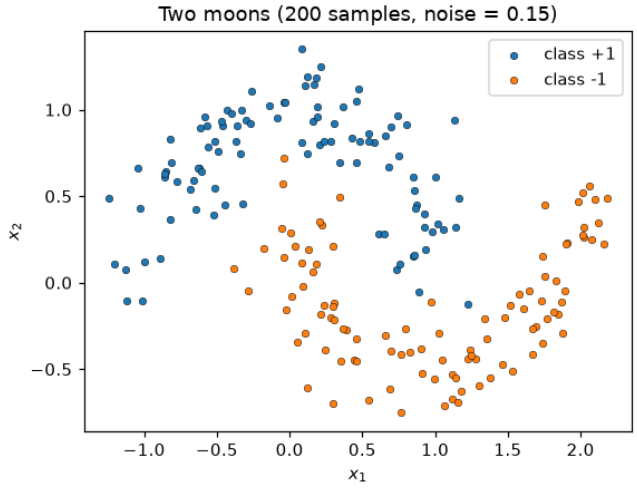


Figure 1: The two-moons dataset: 200 samples with Gaussian noise $\sigma = 0.15$, balanced between the two classes.

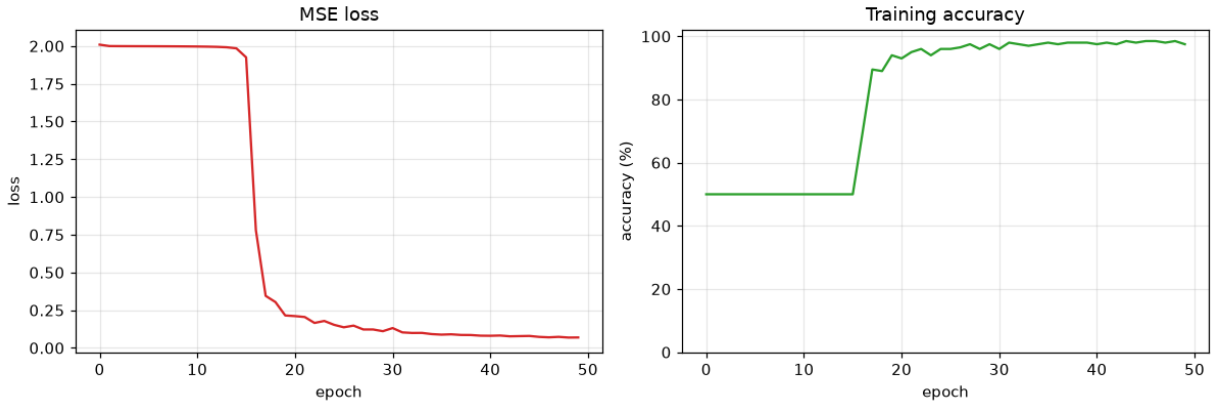


Figure 2: Left: mean squared error per epoch. Right: training accuracy. Both are monotone after the model escapes the initial plateau.

Epoch	Loss	Accuracy	Learning rate
0	2.0090	50.0%	0.0980
10	1.9966	50.0%	0.0801
20	0.2109	93.0%	0.0654
30	0.1316	96.0%	0.0535
40	0.0805	97.5%	0.0437
49	0.0693	97.5%	0.0364

Table 1: Training log (selected epochs) from the executed notebook.

The plateau is `tanh` saturation. The loss sits at ≈ 2.0 for the first $\sim$15 epochs while accuracy stays at chance level (50%). This is not a bug: with tanh outputs and targets ± 1, a wrongly-signed unit starts saturated, and its derivative $1 - t^2$ is then ≈ 0, so it receives almost no gradient. The plateau ends when accumulated mini-batch updates move the pre-activations back into the responsive region — visible as the cliff between epochs 15 and 20 (Figure 2). Mini-batches (rather than one full-batch step per epoch) and the decaying learning rate are what make that escape happen in reasonable wall-clock time.

Outcome. Loss falls from 2.0090 to 0.0693 while accuracy rises from 48–50% (random) to **97.5%**; the boundary in Figure 3 follows both arcs. Accuracy flattens at 97.5% while loss keeps dropping, because MSE on a bounded tanh output keeps refining scores on already-correct points rather than flipping classifications.

Cost. The engine is pure Python: 50 epochs of 200 points (10 batches each) take roughly a minute, and the 60×60 boundary grid costs about 6 seconds of `Value` forward passes. This is the expected price of a scalar engine and is not a concern at this scale.

9 Answers to the conceptual questions

The template repository does not list explicit questions; the set below covers the concepts the assignment description emphasises (chain rule, reverse mode, topological sorting, diamonds, design

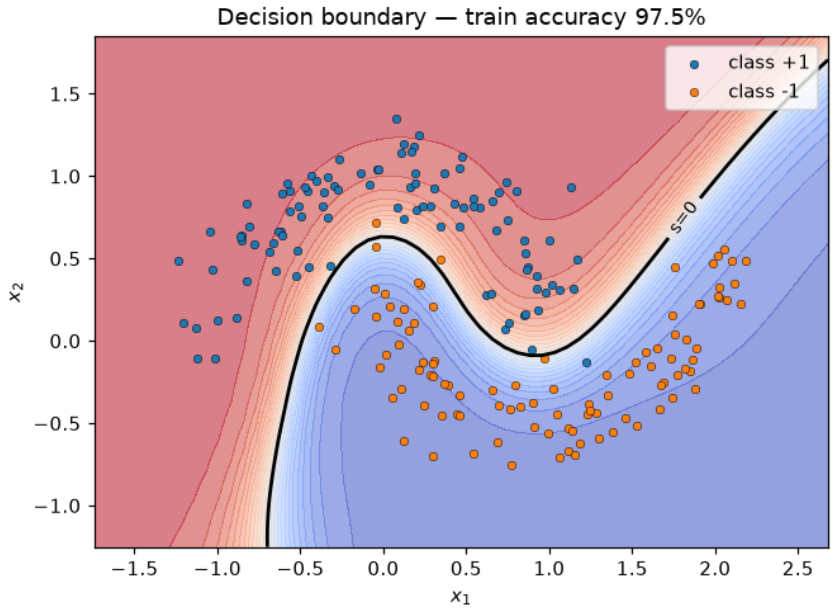


Figure 3: Decision boundary of the trained network. The black contour $s(x) = 0$ separates the two arcs and tracks them closely.

choices, NN API). Additional questions published on the LMS should be appended here.

1. **Why does the gradient flow from the output to the inputs?** Because L is a composition of local functions; Eq. (1) is a recursion from L downwards. Seeding $\partial L/\partial L = 1$ and visiting nodes in reverse topological order evaluates that recursion once, giving every parameter's gradient in a single $\mathcal{O}(|V|)$ sweep.

2. **Why is a topological sort needed, and why post-order?** A node may only be backpropagated through after all consumers of its gradient have contributed; otherwise it would push an incomplete gradient. Post-order DFS lists every node after its dependencies, so reversing that list gives a valid processing order starting from the output.

3. **Why the `visited` set?** DAGs share nodes. Without it a shared node is re-explored once per path (exponential in depth) and appended to the topo list multiple times, so its outgoing gradient is applied several times. See Section 4.

4. **Why must local backward rules use `+=`?** Because a node can receive gradient from several parents, and from several operands of the same operation as in $a + a$; Eq. (2) is a *sum* over paths. Assignment lets the last path overwrite the earlier ones: for $d = (a + a) \cdot a$ at $a = 3$ we would not obtain 12.

5. **Why seed the root's gradient with `1.0`?** We differentiate L with respect to itself, $\partial L/\partial L = 1$; everything else follows from the chain rule. Seeding an intermediate node instead yields that node's gradient with respect to the inputs, which is what makes intermediate-value tests possible.

6. **What would change for a vector-valued output?** Reverse mode gives one output's gradient per sweep; a Jacobian with m outputs needs m backward passes (or a framework with vector–Jacobian products). Training keeps a scalar loss precisely so one pass suffices.

7. **Why is `pow` restricted to constant exponents?** $\partial_a a^k = ka^{k-1}$ holds only for constant k; a `Value` exponent would add $a^k \ln a \cdot \partial k/\partial a$, which the local rule does not implement. An assertion fails fast instead of returning a silently wrong gradient.

8. **Why uniform weights in $[-1, 1]$?** Keeps initial pre-activations in `tanh`'s linear region ($1 - \tanh^2 \approx 1$) so every unit can learn from step 1, and random values break unit symmetry. See Section 5.

9. **Why `tanh` instead of sigmoid?** Zero-centred outputs in $(-1, 1)$ give better-conditioned gradients for downstream layers, and the derivative $1 - t^2$ reuses the forward value. The trade-off is saturation, which the demo's initial plateau illustrates directly.

10. **How does this compare to PyTorch?** PyTorch follows the same two-phase protocol (record the graph in forward, walk it in reverse) but on tensors with fused C++/CUDA kernels, a dynamic autograd graph rebuilt each iteration, and `Tensor.grad` accumulation semantically identical to our `+=`. Our `Value` is the scalar special case of a `Tensor`.

11. **Why is NumPy/PyTorch not allowed here?** The point is to internalise the mechanics — computational graphs, local derivatives, topological ordering, accumulation — rather than to obtain gradients from a library. NumPy appears in the notebook only for dataset generation and plotting, never inside `src/`.

10 Reproducibility

```
# tests
python -m pytest                  # 12 passed

# demo: execute every cell top to bottom and rewrite the figures
jupyter nbconvert --to notebook --execute --inplace demo.ipynb

# report
.tools/tectonic main.tex
```

Everything the results depend on is pinned:

- **Seeds** — `random.seed(42)`, `numpy.random.seed(42)` and `default_rng(42)` are set at the top of the notebook, before any model construction or data generation.
- **Data** — the dataset is generated in-notebook, so no external file or download is involved.
- **No hidden state** — the notebook runs start to finish with no manual intervention; its execution counts run $1 \to 17$ in order with no warnings and no errors.
- **Figures** — `report/figures/*.png` are written by the notebook itself, so the report and the notebook cannot silently drift apart.

11 Conclusion

A scalar autograd engine of roughly one hundred lines is enough to train a small network from scratch: local derivatives in closures, a post-order topological sort for sequencing, and `+=` accumulation for the sum over paths. The three levels of verification — 12/12 tests, a hand-derived sanity

check, and finite differences agreeing to 2.96×10^{-9} — each catch a different class of error, and the end-to-end run reaches 97.5% accuracy on the two-moons task, including a case study of activation saturation.